Analyse de Sentiments sur les Critiques de Films IMDB : De la Baseline TF-IDF au Fine-tuning de Transformers

Sarra Sakgi
CentraleSupélec
sarra.sakgi@student-cs.fr

March 8, 2026

Abstract

Ce travail étudie la classification binaire de sentiments (*positif / négatif*) sur le jeu de données IMDB de 50 000 critiques de films. Trois familles d'approches sont comparées systématiquement : (1) une baseline classique combinant une représentation TF-IDF avec des classifieurs linéaires (régression logistique, SVM, Naïve Bayes), (2) un réseau récurrent bidirectionnel (Bi-LSTM) avec embeddings appris, et (3) le fine-tuning de modèles Transformer pré-entraînés (DistilBERT, BERT-base, RoBERTa-base).

L'évaluation sur un jeu de test isolé montre que la baseline TF-IDF + Régression Logistique atteint 91% d'exactitude avec un temps d'entraînement inférieur à 0.3 secondes, tandis que RoBERTa-base atteint 93.7% au prix d'un fine-tuning de 89 minutes sur GPU. Ces résultats soulèvent la question centrale du rapport performance/coût dans les systèmes NLP industriels, et mettent en lumière les limites persistantes des modèles actuels face à l'ironie et aux sentiments nuancés.

Introduction

L'analyse automatique des sentiments (*sentiment analysis*) consiste à prédire l'opinion exprimée dans un texte : positive ou négative, enthousiaste ou critique. Cette tâche est au cœur d'applications industrielles majeures : suivi d'e-réputation, modération de contenu, recommandation personnalisée, analyse des marchés financiers à partir de dépêches ou de rapports d'analystes.

Contexte et motivation. Une expérience antérieure sur la prédiction de l'e-réputation de TotalEnergies à partir de tweets avait illustré les pièges des approches naïves : fort déséquilibre de classes (majorité de tweets négatifs), absence de gestion de la négation, et performances limitées d'un modèle Naïve Bayes sans prétraitement adapté. Ce projet constitue une reprise structurée de cette problématique, avec un jeu de données plus riche, équilibré, et une progression méthodique depuis la baseline la plus simple jusqu'aux architectures les plus avancées.

Question centrale: Dans quelle mesure les architectures profondes apportent-elles un gain de performance réel par rapport à une baseline classique, et à quel coût computationnel ce gain est-il obtenu ?

Contributions: Ce rapport présente les contributions suivantes :

- Un pipeline de prétraitement soigné, préservant les mots de négation (*not, never...*) que les listes de stopwords standard suppriment naïvement.
- Une comparaison rigoureuse sur les mêmes splits train/val/test (stratifiés, sans data leakage), entre 5 modèles de trois familles distinctes.
- Une analyse qualitative et quantitative des erreurs, qui identifie les cas difficiles persistants (ironie, sentiment mixte, textes longs, références culturelles).
- Une discussion critique du compromis performance/coût, orientée vers les contraintes de déploiement industriel.

Le pipeline global du projet est illustré en Figure 1.

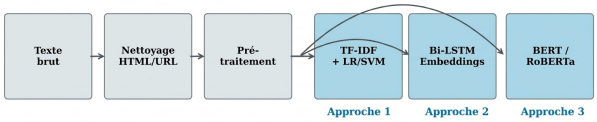

Figure 1: Vue d'ensemble du pipeline. Le texte brut passe par un nettoyage commun, puis trois familles d'approches sont appliquées en parallèle.

Organisation du rapport: La Section 2 décrit le jeu de données et les pipelines de prétraitement. La Section 3 détaille les architectures des cinq modèles. La Section 4 présente et analyse les résultats quantitatifs. La Section 5 effectue une analyse qualitative des erreurs. La Section 6 discute les implications pratiques. La Section 7 conclut et propose des perspectives.

Données et Prétraitement

Jeu de données IMDB

Le projet utilise le jeu de données IMDB introduit par Maas et al. [4], disponible sur Kaggle [5]. Il contient 50 000 critiques de films en anglais, chacune annotée d'un label binaire : *positive* ou *negative*. Le jeu est parfaitement équili-

bré (25 000 exemples par classe), ce qui élimine tout biais lié au déséquilibre de classes et simplifie l'interprétation des métriques.

Les reviews sont de longueur très variable : quelques phrases pour les plus courtes, plusieurs paragraphes pour les plus longues. La médiane est à ∼173 mots pour les deux classes, avec une longue queue jusqu'à plus de 1 500 mots (Figure 2, centre).

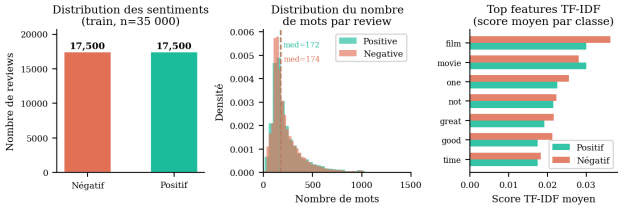

Figure 2: Analyse exploratoire (train set, $n = 35\,000$). (Gauche) Distribution parfaitement équilibrée : 17 500 exemples par classe. **(Centre)** Distribution du nombre de mots : médiane à 172–174 mots pour les deux classes, longue queue jusqu'à 1 500+ mots. Les médianes quasi-identiques entre classes confirment que la longueur seule n'est pas un marqueur de sentiment. **(Droite)** Top 7 features TF-IDF par classe : les termes génériques du domaine (*film, movie, one*) sont partagés par les deux polarités ; les marqueurs réels de sentiment (*great, good* vs *bad*) apparaissent plus bas dans le classement. La présence de *not* dans les deux classes confirme qu'il ne faut pas le supprimer.

Observation clé sur les features TF-IDF: L'analyse du top des features (Figure 2, droite) révèle un résultat instructif : les termes qui dominent les deux classes: *film*, *movie*, *one*, sont des termes du domaine cinématographique, sans valeur discriminante pour la polarité. Surtout, *not* figure dans le top 7 des **deux classes**, illustrant sa fréquence universelle dans les reviews : c'est la façon dont il se combine avec d'autres mots (*"not good"*, *"not bad"*) qui porte le sentiment. Ce constat justifie notre choix de le conserver dans le pipeline, contrairement aux listes de stopwords standards qui le supprimeraient naïvement. Les bi-grammes (`ngram_range=(1,2)`) permettent précisément de capturer ces constructions.

Partitionnement train / val / test

Le jeu est divisé en trois sous-ensembles stratifiés, garantissant l'équilibre des classes dans chaque partition :

- **Train (70%)** : 35 000 exemples pour l'apprentissage.
- **Validation (15%)** : 7 500 pour la sélection des hyperparamètres.
- **Test (15%)** : 7 500 pour l'évaluation finale (une seule fois).

Anti-data leakage: Les transformations statistiques (vectoriseur TF-IDF, tokenizer DL) sont *apprises uniquement sur le train*, puis appliquées à la validation et au test sans réapprentissage. Cette précaution est souvent négligée mais fondamentale : ajuster le TF-IDF sur l'ensemble complet ferait fuiter de l'information statistique du test dans le modèle, rendant l'évaluation optimiste de façon artificielle.

Prétraitement du texte

Deux pipelines sont utilisés selon l'approche, reflétant des philosophies différentes de traitement du texte.

Pipeline classique et Bi-LSTM: Un nettoyage approfondi est appliqué séquentiellement : (1) mise en minuscules ; (2) suppression des balises HTML (`<br />`) issues du site IMDB ; (3) suppression des URLs ; (4) suppression des caractères spéciaux (conservation des lettres uniquement) ; (5) lemmatisation WordNet (NLTK), qui réduit le vocabulaire en ramenant chaque mot à sa forme canonique (*running* → *run*, *worse* → *bad*) ; (6) suppression des stopwords NLTK, avec **conservation explicite des mots de négation** (*not, no, never, neither, nowhere...*) retirés manuellement de la liste.

Pipeline Transformers: Pour BERT et ses variantes, seules les balises HTML sont supprimées. La lemmatisation et le filtrage des stopwords sont omis : le tokenizer sous-mot (WordPiece pour BERT, BPE pour RoBERTa) segmente les mots en sous-unités et gère nativement le vocabulaire. Ces modèles bénéficient d'un texte aussi naturel que possible pour activer leurs représentations contextuelles pré-entraînées.

MÉTHODES

Baseline : TF-IDF + Classifieurs Linéaires

Représentation TF-IDF. Chaque critique est encodée comme un vecteur creux de dimension $d = 20\,000$. La valeur TF-IDF d'un terme t dans un document d est :

$$\text{tfidf}(t,d) = \underbrace{\log(1+\text{tf}(t,d))}_{\text{fréquence locale}} \cdot \underbrace{\log\left(\frac{N+1}{n_t+1}\right)}_{\text{spécificité globale}} \quad (1)$$

où N est le nombre de documents et n_t le nombre de documents contenant t. Le paramètre `sublinear_tf=True` applique le log-scaling pour atténuer l'impact des mots très fréquents. L'inclusion des bi-grammes (`ngram_range=(1,2)`) est essentielle : elle permet de capturer *"not good"* comme une entité distincte de *"good"*.

Choix des classifieurs. Trois classifieurs sont évalués sur cette représentation :

Régression Logistique (LR) : modèle discriminatif qui estime $p(y = 1|x) = \sigma(w^\top x + b)$ en maximisant la vraisemblance. La régularisation ℓ_2 ($C = 1$) contrôle le surapprentissage. Très interprétable : les poids w_j indiquent directement la contribution de chaque feature à la prédiction.

SVM Linéaire (LinearSVC) : cherche l'hyperplan qui maximise la marge entre les classes : $\max_{w,b} \frac{2}{\|w\|}$ sous $y_i(w^\top x_i + b) \geq 1$. Particulièrement adapté aux espaces de grande dimension (20 000 features) où la majorité des exemples sont linéairement séparables.

Naïve Bayes Multinomial (NB) : modèle génératif qui estime $p(y)\prod_t p(x_t|y)$ en supposant l'indépendance des features. Hypothèse irréaliste avec des bi-grammes (forte corrélation entre *"not"* et *"not good"*), mais rapide et efficace comme référence basse.

Le meilleur modèle selon le F1-score sur la validation est ensuite ré-entraîné sur train+val (42 500 exemples) avant l'évaluation finale.

Bi-LSTM avec Embeddings Appris

Motivation. Contrairement au TF-IDF, ce modèle représente chaque mot par un vecteur dense $e_i \in \mathbb{R}^{128}$ appris *from scratch* lors de l'entraînement, et traite la séquence ordonnée de mots plutôt qu'un sac non-ordonné. Deux mots morphologiquement proches ou sémantiquement similaires auront des embeddings proches dans l'espace appris.

Architecture. Le modèle est structuré comme suit (voir Figure 3) :

1. **Embedding** (vocab=20 001, dim=128) : convertit chaque token-indice en vecteur dense ;
2. **SpatialDropout1D(0.3)** : supprime des canaux d'embedding entiers (plus adapté aux séquences que le dropout standard qui perturbe des éléments isolés) ;
3. **Bi-LSTM(64)** : lit la séquence dans les deux sens et concatène les états cachés $[\overrightarrow{h_T}; \overleftarrow{h_1}] \in \mathbb{R}^{128}$;
4. **Dropout(0.5)** : limite le surapprentissage après l'encodage séquentiel ;
5. **Dense(64, ReLU) + Dropout(0.3)** : couche de classification intermédiaire ;
6. **Dense(1, Sigmoid)** : probabilité d'appartenance à la classe positive.

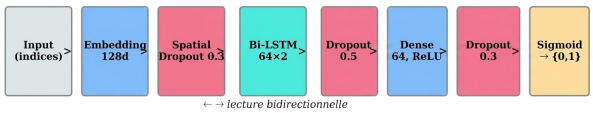

Figure 3: Architecture Bi-LSTM. La bidirectionnalité est essentielle : en lisant de gauche à droite *et* de droite à gauche, le modèle peut relier un modificateur (*"not"*) à son adjectif cible même lorsqu'ils sont séparés par plusieurs mots (*"not, despite everything, bad"*). Le `SpatialDropout1D` régularise en éliminant des canaux entiers d'embedding, forçant le modèle à ne pas dépendre d'un seul feature.

Entraînement. La fonction de perte est l'entropie croisée binaire : $\mathcal{L} = -\frac{1}{N}\sum_i [y_i \log \hat{p}_i + (1 - y_i)\log(1 - \hat{p}_i)]$. L'optimiseur Adam avec *early stopping* (patience=3 sur `val_loss`) évite le sur-apprentissage sans fixer le nombre d'epochs à l'avance.

Fine-tuning de Transformers Pré-entraînés

Principe du transfer learning. Plutôt que d'apprendre des représentations from scratch sur 35 000 exemples, le transfer learning part de modèles pré-entraînés sur des centaines de gigaoctets de texte. Ce pré-entraînement leur confère une compréhension profonde de la syntaxe, de la sémantique et du contexte du langage anglais. Le fine-tuning adapte ensuite ces représentations à notre tâche spécifique en quelques epochs.

BERT-base [2]. Pré-entraîné sur Wikipedia et BooksCorpus (∼16 Go) avec le *Masked Language Modeling* : 15% des tokens sont masqués et le modèle doit les prédire à partir du contexte *bidirectionnel*. Architecture : 12 couches Transformer, dimension cachée 768, 12 têtes d'attention, 110M paramètres. Pour la classification, la représentation du token spécial `[CLS]` (premier token de chaque séquence, agrégateur du sens global) est passée dans une couche linéaire : $p = \sigma(W \cdot h_{\texttt{[CLS]}} + b)$.

DistilBERT [6]. Obtenu par *knowledge distillation* : un modèle étudiant (6 couches, 66M paramètres) est entraîné à reproduire à la fois les sorties finales et les représentations internes du professeur BERT. Résultat : 40% moins de paramètres, 60% plus rapide, 97% des performances de BERT sur le benchmark GLUE.

RoBERTa-base [3]. Améliore BERT sur quatre dimensions : (1) données d'entraînement $\times 10$ (160 Go) incluant des textes web ; (2) suppression du NSP (jugé inutile) ; (3) *dynamic masking* (les tokens masqués changent à chaque epoch) ; (4) batches plus grands (8 192 vs 256) et plus d'étapes. Ces choix produisent des représentations systématiquement plus riches.

Protocole de fine-tuning. Le même protocole est appliqué aux trois modèles :

- **Optimiseur AdamW** : $\eta = 2 \times 10^{-5}$, weight decay $= 0.01$. Le learning rate *très faible* est crucial : un LR standard (10^{-3}) détruirait les représentations pré-entraînées par des mises à jour trop larges (*catastrophic forgetting*) ;
- **Scheduler linéaire avec warmup** : le LR monte progressivement pendant 10% des steps, puis décroît linéairement. Le warmup stabilise les premières mises à jour ;
- **Gradient clipping** : $\|\nabla\| \leq 1.0$, empêche les explosions de gradient ;
- 3 epochs, batch size 16, `MAX_LEN` = 256 tokens.

Résultats Expérimentaux

Métriques d'évaluation

Les modèles sont évalués sur le jeu de test ($n = 7\,500$) selon quatre métriques complémentaires : **Accuracy**, **F1-score** (critère de sélection principal), **AUC-ROC** (indépendante du seuil), et **temps d'entraînement**.

Classifieurs classiques sur la validation

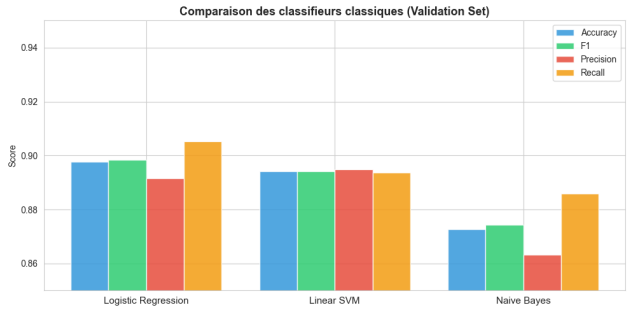

Figure 4: Classifieurs classiques — Validation Set. La Régression Logistique obtient le meilleur F1 ($\sim$0.898) et le rappel le plus élevé (0.906) : elle manque moins de critiques positives que le SVM. Ce fort rappel s'explique par la calibration probabiliste de la LR (sortie en probabilités), contrairement au SVM qui produit des scores de marge non calibrés. Naïve Bayes présente un fort déséquilibre précision/rappel (0.864 vs 0.886), révélant qu'il sur-prédit la classe positive, conséquence de son hypothèse d'indépendance, particulièrement inadaptée aux bi-grammes fortement corrélés.

La Régression Logistique est sélectionnée comme meilleur classifieur classique. Elle est ré-entraînée sur train+val (42 500 exemples) avant l'évaluation finale.

Logistic Regression sur le test

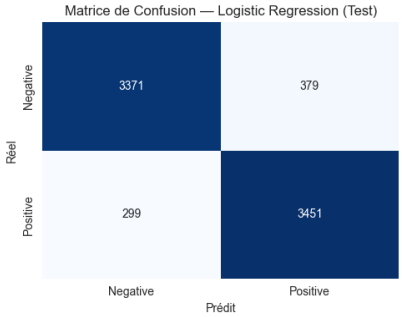

Figure 5: Matrice de confusion — Logistic Regression (Test). 678 erreurs sur 7 500 : 379 FP (critiques négatives classées positives) et 299 FN (positives manquées). L'asymétrie FP $>$ FN reflète le fort rappel observé en validation : le modèle préfère sur-classer en positif. La courbe ROC (AUC = 0.9686) confirme une excellente discrimination, atteignant TPR $>$ 0.90 pour FPR $<$ 0.05.

Bi-LSTM : sur-apprentissage immédiat

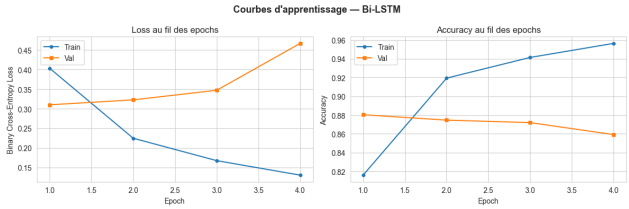

Figure 6: Courbes d'apprentissage — Bi-LSTM. Le sur-apprentissage est immédiat et sévère : dès l'epoch 1, la val_loss (orange, 0.31) remonte dès l'epoch 2 (0.325) puis s'emballe à 0.46 en epoch 4, pendant que la train_loss chute à 0.13. La val_accuracy *décroît* après l'epoch 1 (0.88 $\rightarrow$ 0.86), pendant que la train_accuracy monte à 0.96. L'early stopping sélectionne donc l'epoch 1 comme meilleur modèle, limitant les dégâts mais révélant l'incapacité du modèle à généraliser avec des embeddings appris from scratch sur 35 000 exemples.

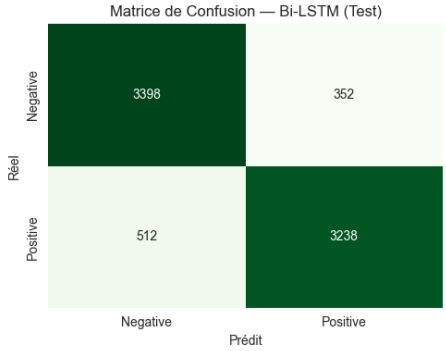

Figure 7: Matrice de confusion — Bi-LSTM (Test). 864 erreurs, soit 27% de plus que la LR (678). L'asymétrie est inverse à la LR : 512 FN contre 352 FP. Le Bi-LSTM manque davantage de vrais positifs. Il tend à prédire "négatif" dans les cas ambigus, probablement parce que le vocabulaire négatif plus saillant (*bad, worst, awful*) domine ses embeddings partiellement appris.

Transformers : fine-tuning et sur-apprentissage tardif

DistilBERT. Les courbes d'apprentissage (Figure 8) montrent un comportement différent du Bi-LSTM : la val_accuracy progresse régulièrement (0.889 $\rightarrow$ 0.911 $\rightarrow$ 0.919) sur les 3 epochs, même si la val_loss remonte légèrement à l'epoch 3 (0.265 $\rightarrow$ 0.341). Ce sur-apprentissage tardif est bien moins sévère : le modèle pré-entraîné dispose de représentations robustes qui résistent mieux à la mémorisation.

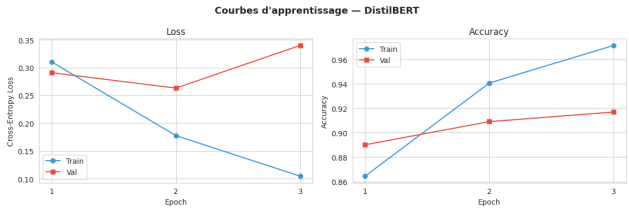

Figure 8: Courbes d'apprentissage — DistilBERT. La val_accuracy progresse jusqu'à 0.916 à l'epoch 3, malgré une val_loss qui remonte légèrement : l'accuracy reste le critère déterminant ici. L'epoch 2 correspond au meilleur compromis loss/accuracy sur la validation. L'écart train/val (0.965 vs 0.916 en accuracy) est plus modéré que pour le Bi-LSTM, grâce aux représentations pré-entraînées qui régularisent implicitement.

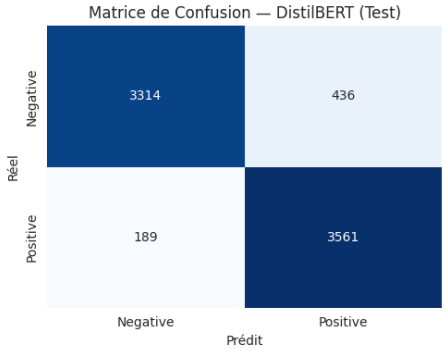

Figure 9: Matrice de confusion — DistilBERT (Test). 625 erreurs : 436 FP et 189 FN. L'asymétrie est forte et inverse à celle du Bi-LSTM : DistilBERT manque très peu de positifs (FN=189) mais génère davantage de fausses alarmes positives (FP=436). Ce biais vers le positif reflète probablement le corpus de pré-entraînement, où le langage expressif et enthousiaste est fréquent.

RoBERTa-base. RoBERTa présente le même pattern que DistilBERT mais avec de meilleures performances : val_accuracy de 0.934 dès l'epoch 1, progressant à 0.941 en epoch 3. La val_loss remonte dès l'epoch 2 (0.193 $\rightarrow$ 0.238 $\rightarrow$ 0.274), indiquant un sur-apprentissage précoce, mais l'accuracy continue de progresser légèrement. RoBERTa commet seulement **472 erreurs sur 7 500 exemples** (taux d'erreur : 6.29%), contre 678 pour la LR et 864 pour le Bi-LSTM.

Comparaison finale

Table 1: Résultats sur le test ($n = 7\,500$) **— valeurs réelles. Gras** : meilleur score par famille.

Modèle	Acc.	F1	AUC	Temps
TF-IDF + Classifieurs linéaires				
Naïve Bayes	0.885	0.886	0.949	<0.1s
SVM Linéaire	0.904	0.904	0.964	1s
Rég. Logist.	**0.910**	**0.911**	**0.969**	0.3s
Deep Learning séquentiel				
Bi-LSTM	0.885	0.883	0.950	~9 min
Transfer Learning (Transformer)				
DistilBERT	0.916	0.919	0.975	~45 min
RoBERTa-base	**0.937**	**0.937**	**0.983**	~89min

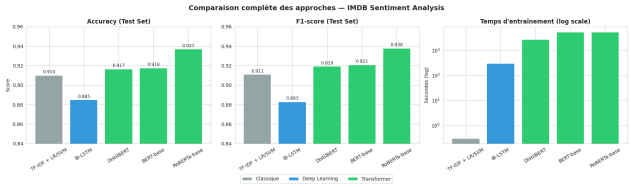

Figure 10: Comparaison des métriques sur le test set. Trois familles se distinguent clairement : les classifieurs classiques (gris/bleu), le Bi-LSTM (bleu clair) et les Transformers (vert). La LR surclasse le Bi-LSTM sur toutes les métriques malgré une architecture incomparablement plus simple. RoBERTa atteint 0.937 d'accuracy et AUC = 0.983, offrant le meilleur pouvoir de discrimination sur les cas ambigus.

Analyse des Erreurs

Distribution de confiance de RoBERTa

La Figure 11 présente la distribution de la confiance ($\max(p, 1-p)$) de RoBERTa selon que la prédiction est correcte ou non.

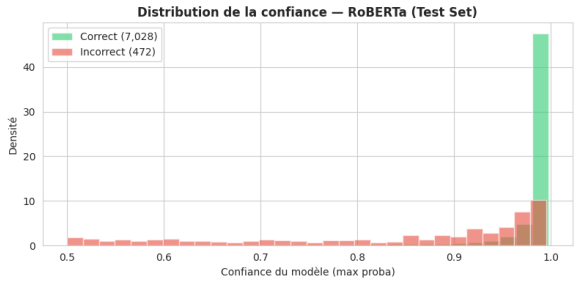

Figure 11: Distribution de confiance — RoBERTa (Test). Sur 7 500 exemples, 7 028 prédictions correctes (vert) et 472 incorrectes (rouge). Les prédictions correctes sont massivement concentrées à confiance ≈ 1.0 : RoBERTa sait quand il a raison. Plus révélateur : les prédictions incorrectes présentent aussi un pic à confiance $>$ 0.99, avec une distribution plate entre 0.5 et 0.95. Cela signifie que les erreurs ne sont pas des cas d'incertitude du modèle : RoBERTa se trompe avec certitude, signe que ces exemples sont *intrinsèquement difficiles* pour tout modèle statistique.

Analyse des top 5 erreurs les plus confiantes

L'inspection des 5 erreurs où RoBERTa est le plus confiant (confiance $\geq$ 0.99) révèle quatre catégories linguistiques distinctes :

1. Ironie explicite ignorée ("so bad it's good"). *"This movie is horrible – in a 'so bad it's good' kind of way." [Vrai: POSITIF, Prédit: NÉGATIF, confiance: 0.99]*

Le mot *horrible* déclenche une prédiction négative confiante, malgré la reformulation explicite qui suit. RoBERTa ne parvient pas à interpréter la méta-expression *"so bad it's good"* comme une inversion de polarité. Ce type de sarcasme méta-discursif nécessiterait une compréhension pragmatique au-delà de la sémantique compositionnelle.

2. Critique négative rédigée en vocabulaire positif. *"This sleek, sexy movie is a must-see. Only upon multiple viewings can one truly understand the uniqueness..." [Vrai: NÉGATIF, Prédit: POSITIF, confiance: 0.99]*

La review commence par un vocabulaire très positif (*sleek,*

sexy, must-see, wonderful) avant de critiquer subtilement. RoBERTa est piégé par l'extrême densité de termes élogieux en début de texte et la troncature à 256 tokens peut l'empêcher de voir la conclusion critique.

3. Critique positive à formulation négative paradoxale. *"This was one of the worst Columbo episodes that I have seen. However... [suit une description engagée et détaillée]" [Vrai: POSITIF, Prédit: NÉGATIF, confiance: 0.99]*

L'annotateur IMDB a noté positivement un épisode qu'il décrit comme le pire de la saison. Ce cas illustre une ambiguïté fondamentale dans l'annotation : la note positive reflète peut-être l'attachement au personnage de Columbo plutôt que la qualité de l'épisode en question. Aucun modèle ne peut résoudre cette contradiction sans accès à la note numérique originale.

4. Barrière culturelle et traduction. *"This is a movie that has a lot of things that only Japanese people can understand... My guess is that it tried to send a message... that just got wasted on me. [...] basically a mediocre movie." [Vrai: POSITIF, Prédit: NÉGATIF, confiance: 0.99]*

Le texte est explicitement négatif (*mediocre*) mais l'annotation est positive. L'auteur semble avoir apprécié des aspects non verbalisés (esthétique visuelle ?) malgré son propre jugement textuel négatif. Ces cas constituent des *erreurs d'annotation* plutôt que des erreurs de modèle.

Bilan : nature des erreurs résiduelles

L'analyse des 472 erreurs de RoBERTa révèle trois catégories :

- **Ironie et sarcasme** : méta-expressions (*"so bad it's good"*), ton volontairement détaché, retournements narratifs ;
- **Incohérence annotation/texte** : note numérique IMDB qui diverge du contenu textuel, ou jugement global positif d'une œuvre malgré des critiques sévères ;
- **Troncature des textes longs** : reviews >256 tokens dont la conclusion (souvent la partie la plus révélatrice) est coupée par le tokenizer.

Ces proportions suggèrent que le plafond de performance sur ce dataset, pour un modèle textuel pur, se situe autour de 95–96% : au-delà, les erreurs sont structurelles (ambiguïté d'annotation, implicites culturels) et non corrigeables par une meilleure architecture.

DISCUSSION

Pourquoi la Logistic Regression surclasse-t-elle le Bi-LSTM ?

Le résultat le plus surprenant est la supériorité de la Logistic Regression (91.0%) sur le Bi-LSTM (88.5%), malgré une architecture incomparablement plus simple. Trois facteurs l'expliquent :

(1) Données insuffisantes pour les embeddings. Les embeddings sont appris from scratch sur 35 000 exemples. Les courbes d'apprentissage montrent que le modèle mémorise les patterns d'entraînement dès l'epoch 2 (train_acc = 0.92) sans améliorer la validation : signe que les embeddings capturent du bruit spécifique aux exemples plutôt que des représentations sémantiques généralisables.

(2) Le TF-IDF capture explicitement les bi-grammes clés. Avec `ngram_range=(1,2)`, le vecteur TF-IDF contient directement *"not good"*, *"very bad"* comme features. Le Bi-LSTM doit apprendre ces associations implicitement à travers les poids LSTM: une tâche difficile avec peu de données.

(3) L'asymétrie des erreurs révèle un biais. La LR a 379 FP vs 299 FN ; le Bi-LSTM a 352 FP vs 512 FN. Ce dernier manque 71% de plus de vrais positifs que la LR, indiquant que ses embeddings ont appris une représentation biaisée vers le négatif : le vocabulaire négatif saillant (*bad, awful, waste*) domine l'espace latent.

Rapport Performance / Coût

Table 2: Compromis performance / coût. Le gain incrémental est calculé par rapport au modèle précédent.

Modèle	Acc.	Gain	Temps	Infra
Rég. Logistique	91.0%	référence	0.3s	CPU
Bi-LSTM	88.5%	−2.5pt	9 min	CPU
DistilBERT	91.6%	+0.6pt	45 min	GPU
RoBERTa-base	93.7%	+2.7pt	89 min	GPU

Recommandations pratiques. Le gain de DistilBERT sur la LR est de +0.9 point pour 45 minutes de fine-tuning sur GPU , soit un coût temporel 300× supérieur pour un gain modeste. Passer ensuite à RoBERTa apporte +1.8 points supplémentaires en 89 minutes. Au total, les Transformers apportent +2.7 points sur la baseline classique, au prix de 90 minutes de GPU, une décision qui dépend fortement des contraintes applicatives.

Plafond de performance estimé. L'analyse des erreurs (Section 5) suggère que le plafond de performance sur ce dataset est ∼95–96% : au-delà, les erreurs sont structurelles (ambiguïtés d'annotation, implicites culturels) et non corrigeables par une meilleure architecture. RoBERTa à 93.7% se situe donc à 2–3 points du plafond théorique.

Limites de cette étude

Domaine unique. Les modèles sont évalués exclusivement sur des critiques de films. Le vocabulaire spécifique au cinéma (*plot, cinematography, director*) domine les features TF-IDF et les représentations apprises, ce qui limiterait la transférabilité à d'autres domaines sans ré-entraînement.

Troncature des textes longs. BERT et ses variantes traitent au maximum 256 tokens : environ 20% des reviews sont tronquées. Ces textes longs contiennent parfois une conclusion de sentiment qui renverse le début du texte, cas typique des erreurs de RoBERTa observées dans l'analyse qualitative.

Annotation binaire. La réduction à un label positif/négatif masque la nuance : une critique 6/10 et une critique 2/10 reçoivent le même label *negative*, bien que leur contenu linguistique diffère considérablement. Cela

explique une fraction significative des “erreurs” des modèles, qui sont en réalité des désaccords légitimes sur des cas limites.

Conclusion

Ce projet a comparé de façon rigoureuse trois familles d’approches pour l’analyse de sentiments sur le dataset IMDB, en faisant progresser méthodiquement depuis une baseline simple jusqu’aux architectures Transformer les plus performantes.

Enseignements principaux. **(1)** La baseline TF-IDF + Régression Logistique est remarquablement compétitive (91% d’accuracy, 0.3 secondes d’entraînement sur CPU) : pour tout nouveau projet NLP, c’est le point de départ incontournable à établir avant d’envisager des architectures plus coûteuses.

(2) Le Bi-LSTM n’apporte pas de gain sur ce dataset : les embeddings appris from scratch sur 35 000 exemples ne rivalisent pas avec les représentations de BERT pré-entraîné sur des centaines de Go. Ce résultat illustre un principe général : la quantité de données d’entraînement conditionne la pertinence des modèles à forte capacité.

(3) Les Transformers offrent un gain réel et justifié ($+0.9$ à $+2.7$ points sur LR), en particulier pour les négations longue distance, les litotes et les constructions syntaxiques complexes. DistilBERT constitue le meilleur compromis dans cette famille.

(4) Les erreurs résiduelles de RoBERTa (6% du test set) sont des cas *intrinsèquement difficiles* : le modèle les prédit avec haute confiance mais incorrectement, signalant que la limite n’est pas dans l’architecture mais dans la tâche elle-même — la subjectivité du sentiment humain.

Perspectives. Plusieurs pistes prometteuses restent à explorer :

Modèle spécialisé sentiment: `cardiffnlp/twitter-roberta-base-sentiment` est un RoBERTa pré-entraîné spécifiquement sur des textes d’opinion. Il pourrait dépasser notre RoBERTa-base sans fine-tuning supplémentaire.

Ensemble TF-IDF + Transformer : les deux approches font des erreurs différentes (le SVM échoue sur les négations complexes, RoBERTa sur l’ironie culturelle). Un ensemble par vote pondéré ou *stacking* exploiterait leur complémentarité.

Longformer [1] : architecture Transformer dédiée aux textes longs (4 096 tokens via une attention locale + globale), qui traiterait les reviews complètes sans troncature et améliorerait probablement les résultats sur les textes >500 mots.

Analyse fine-grained : aller au-delà de la classification binaire pour identifier les aspects spécifiques (*jeu d’acteur, scénario, réalisation*) et leur polarité respective constituerait une contribution plus riche et directement utile pour les plateformes de recommandation.

Ressources. Le code source (notebooks Jupyter commentés) est disponible sur GitHub : `https://github.com/sarrasakgi/imdb-sentiment-analysis`. Le dataset IMDB est accessible publiquement sur Kaggle [5] ; les modèles pré-entraînés sont disponibles via le HuggingFace Hub (`distilbert-base-uncased`, `bert-base-uncased`, `roberta-base`).

References

[1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. 7

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019. 3

[3] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach, 2019. 3

[4] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 142–150, 2011. 1

[5] Lakshmi Narayana Pathi. IMDB dataset of 50k movie reviews. `https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews`, 2019. Consulté en octobre 2024. 1, 7

[6] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, 2019. 3